CITY CARE

Documentación completa 2.6 Frontend

Integración del panel administrativo con el backend y el cliente Orval

Este documento explica qué se implementó en el frontend definitivo de City Care, cómo funciona la sesión, cómo se protegen las rutas por rol y cómo el panel de usuarios consume las operaciones del backend. El resultado es un apartado 2.6 cerrado y verificable, sin modificar el alcance futuro de incidencias y equipos.

Dato	Resultado
Proyecto frontend	C:\Users\migue\OneDrive\Desktop\front-medular\mi-proyecto
Backend	C:\Users\migue\medula\city-care
Estado del apartado	Completo 9 de 9
Fecha de corte	31 de octubre de 2026

Conclusión

El frontend ya no es solamente una maqueta. El administrador puede consultar, filtrar, paginar, crear y editar usuarios, cambiar su estado y restablecer contraseñas. Las rutas están protegidas por sesión y rol, el token se adjunta de forma centralizada y la interfaz comunica carga, resultados vacíos, éxito, error y confirmaciones.

1 Alcance y estado final

El apartado 2.6 se limita al frontend administrativo disponible en esta etapa. La vista de administración de dependencia conserva su diseño, navegación protegida y datos de sesión reales; la conexión de incidencias y equipos se realizará cuando existan los módulos de backend correspondientes.

Punto	Estado	Evidencia
Vista del panel administrador	Completo	Página /admin/users integrada en Nuxt.
Vista de alumbrado público	Completo	Página visual protegida para administrador de dependencia.
Manuales visuales	Completo	Documentación de ambas vistas disponible.
Frontend definitivo localizado	Completo	Proyecto original front-medular/mi-proyecto.
Vistas integradas	Completo	Layouts, barras, tablas y componentes forman parte del proyecto.
Panel de usuarios conectado	Completo	Consume las funciones tipadas generadas por Orval.
Rutas protegidas por rol	Completo	Middlewares admin, dependency-admin y guest.
Sesión administrada	Completo	Login, restauración, Bearer y logout centralizados.
Estados de interfaz	Completo	Carga, vacío, éxito, error, reintento y confirmación.

2 Arquitectura de integración

La interfaz no construye solicitudes HTTP manuales en cada componente. Las operaciones OpenAPI generadas se concentran en servicios de dominio y todas pasan por un cliente común que configura la URL y el encabezado Bearer.

Capa	Responsabilidad	Ubicación principal
Página	Coordina filtros, paginación, modales, avisos y actualización de datos.	app/pages/admin/users.vue
Componentes	Presentan formulario, tabla, barra de herramientas, navegación y encabezado.	app/components/admin
Servicio	Expone operaciones de usuarios, roles y dependencias con nombres de dominio.	app/services/users/users.service.ts
SDK Orval	Contiene tipos y funciones generadas desde OpenAPI.	app/generated/api/city-care.ts
Cliente HTTP	Resuelve la URL del backend, agrega Bearer y normaliza errores.	app/services/api/api.client.ts
Sesión	Guarda token, restaura perfil y ejecuta logout.	app/composables/useAuth.ts
Middleware	Autoriza la navegación según sesión y rol.	app/middleware

3 Panel administrativo de usuarios

3.1 Consulta y filtros

Al abrir el panel se solicitan los usuarios, roles y dependencias activas. La búsqueda por texto utiliza un retraso breve para evitar consultas por cada tecla. Los filtros de rol y estado reinician la paginación y vuelven a cargar los datos. El resumen superior muestra totales, cuentas activas y administradores.

- La tabla usa el tipo UserPublic generado por el contrato OpenAPI.
- La paginación usa la metadata entregada por el backend.
- El estado vacío distingue entre falta de registros y filtros sin resultados.

3.2 Alta de usuario

El botón Nuevo usuario abre un formulario con nombre, apellido, correo, dependencia, rol, teléfono, cargo y contraseña. Antes de enviar se validan los campos obligatorios y la coincidencia de contraseña. El backend conserva la validación definitiva y la transacción entre users y credentials.

3.3 Edición

La acción Editar reutiliza el formulario y precarga los datos disponibles. En este modo no se solicita contraseña; el cambio de contraseña se mantiene como una operación separada para conservar una intención clara y revocar sesiones.

3.4 Activación y baja lógica

La interfaz nunca ofrece eliminación física. Activar o desactivar requiere confirmación, llama al endpoint de estado y actualiza el listado. Al desactivar, el backend revoca las sesiones del usuario; la interfaz informa esta consecuencia.

3.5 Restablecimiento de contraseña

La acción solicita una nueva contraseña y confirmación. Después de una respuesta satisfactoria, se muestra un aviso y el backend invalida las sesiones anteriores para que el usuario vuelva a autenticarse.

Acción visual	Operación del SDK	Ruta del backend
Cargar tabla	listUsers	GET /api/v1/admin/users
Crear	createUser	POST /api/v1/admin/users
Editar	updateUser	PUT /api/v1/admin/users/{id}
Activar o desactivar	updateUserStatus	PATCH /api/v1/admin/users/{id}/status
Nueva contraseña	resetUserPassword	PATCH /api/v1/admin/users/{id}/password
Catálogo de roles	listRoles	GET /api/v1/admin/roles
Catálogo de dependencias	listDependencies	GET /api/v1/admin/dependencies

4 Sesión y seguridad de navegación

4.1 Token y Bearer

El token se conserva en la cookie city-care-token y no se replica en componentes. El cliente HTTP central consulta la cookie y agrega Authorization Bearer automáticamente. Esto reduce omisiones y mantiene las funciones generadas sin lógica repetida.

4.2 Restauración de perfil

Cuando existe token pero todavía no hay usuario en memoria, restoreSession consulta el perfil. Si la sesión dejó de ser válida, el estado local se limpia. Las barras superiores muestran el nombre, rol e iniciales del usuario autenticado.

4.3 Matriz de navegación

Ruta o escenario	Sin sesión	admin	dependency_admin	Otros roles
/	Permanece en login	Redirige a /admin/users	Redirige al panel de dependencia	Permanece en login
/admin/users	Redirige a /	Acceso permitido	Redirige al panel de dependencia	Redirige a /
/dependencia/alumbrado-publico	Redirige a /	Acceso permitido	Acceso permitido	Redirige a /

4.4 Cierre de sesión

Los menús administrativos incluyen un botón de salida con estado de carga. Primero se solicita al backend eliminar el token actual; después se limpia el token y el perfil local y se navega al login.

5 Estados y comunicación con el usuario

Estado	Comportamiento
Carga	Desactiva controles relevantes y muestra que la información está en proceso.
Vacío	Explica que no hay usuarios o que los filtros no produjeron resultados.
Éxito	Confirma altas, ediciones, cambios de estado y contraseña.
Error	Muestra un mensaje comprensible y conserva la oportunidad de reintentar.
Confirmación	Solicita aprobación antes de activar o desactivar una cuenta.
Operación ocupada	Impide dobles envíos mientras una mutación está en curso.

6 Vista de administración de dependencia

La vista de Alumbrado Público conserva el esqueleto visual aprobado: resumen, equipos e incidencias. En el 2.6 se integró al proyecto definitivo, se protegió su navegación y se conectaron los datos de sesión y el logout. Los datos operativos siguen siendo demostrativos porque los módulos de incidencias y equipos pertenecen a los apartados 3.1 y 3.2.

7 Archivos implementados o actualizados

Archivo	Función
app/pages/admin/users.vue	Orquesta el panel conectado.
app/components/admin/AdminUserToolbar.vue	Búsqueda y filtros controlados.
app/components/admin/AdminUserTable.vue	Tabla real, estados y paginación.
app/components/admin/AdminUserFormModal.vue	Formulario de alta y edición.
app/services/users/users.service.ts	Servicio sobre las funciones Orval.
app/middleware/admin.ts	Protección de administrador general.
app/middleware/dependency-admin.ts	Protección del panel de dependencia.
app/middleware/guest.ts	Evita mostrar el login a una sesión ya iniciada.
app/composables/useAuth.ts	Estado de sesión compartido.
app/components/admin/AdminSidebar.vue	Logout administrativo.
app/components/dependency/DependencySidebar.vue	Logout de dependencia.
app/components/admin/AdminTopbar.vue	Identidad real del administrador.
app/components/dependency/DependencyTopbar.vue	Identidad real del usuario de dependencia.
app/assets/css/admin.css	Estados, modal y adaptación responsive.
app/assets/css/dependency-admin.css	Estado visual del logout.

8 Verificación realizada

Prueba	Resultado	Evidencia
Formato	Aprobado	Prettier aplicado a los archivos modificados.
Tipos	Aprobado	npm run typecheck sin errores.
Compilación	Aprobado	npm run build genera el cliente y servidor Nuxt.
API con administrador	Aprobado	Login 200; perfil, usuarios, roles y dependencias accesibles.
Ruta sin sesión	Aprobado	/admin/users redirige al login.
Ruta con sesión	Aprobado	/admin/users responde 200 y presenta el panel.

9 Operación y mantenimiento

Para ejecutar el frontend durante desarrollo:

- Iniciar el backend city-care en http://localhost:3333.
- En el proyecto mi-proyecto ejecutar npm run dev.
- Verificar NUXT_PUBLIC_API_BASE_URL cuando el backend use otra dirección.
- Después de cambiar OpenAPI, regenerar el SDK con el comando configurado y ejecutar typecheck y build.

10 Criterio de cierre

El apartado 2.6 se considera terminado porque sus nueve criterios están implementados y comprobados. El siguiente trabajo técnico no es completar este frontend, sino formalizar el cierre del repositorio y después diseñar los modelos, migraciones y endpoints de incidencias y equipos.